

Quantara Vultra — Pipeline v2 (Revised)

What changed from v1

Added **stocks** alongside crypto (multi-asset, not crypto-only)

Replaced "UI automation" idea with **direct broker APIs**: ccxt (crypto) + Alpaca (stocks) — consistent speed, no ToS risk, no 30-50s webhook latency

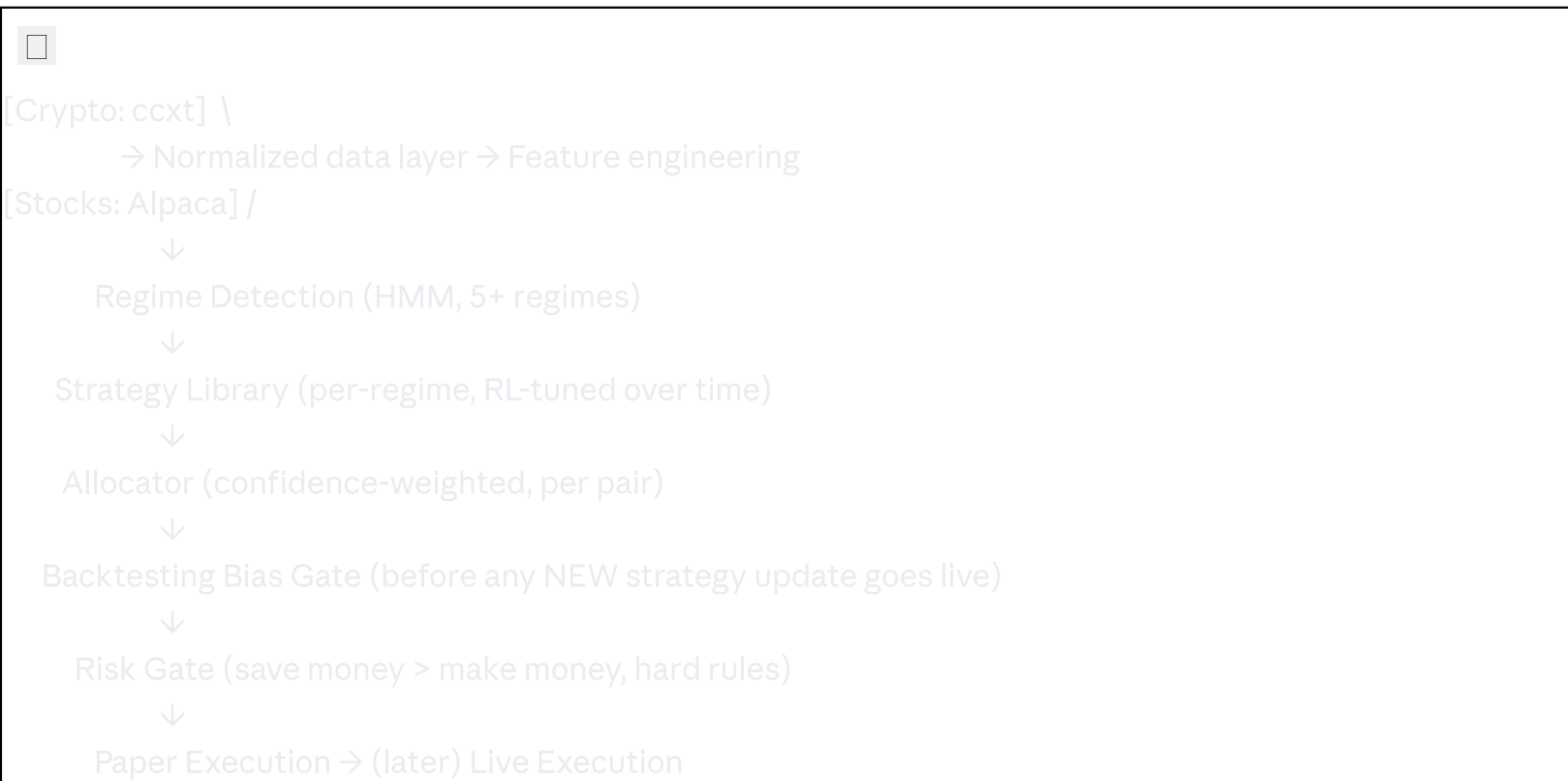
Added a **reinforcement learning layer** that improves strategy performance over time

Added a **backtesting bias validation gate** — no strategy goes live until it passes checks against 6 known biases

Confirmed **parallel scanning**: continuously evaluate all pairs/timeframes across both asset classes, deploy where edge is found, keep scanning the rest

Risk philosophy unchanged and still central: **save money > make money**

Revised High-Level Flow



Everything from v1 (schema, adapters, aggregator, storage, feature engineering, HMM regime model) stays the same structurally — Alpaca just becomes a second adapter alongside Binance/Coinbase, implementing the same `BaseAdapter` interface. That's the payoff of having built that abstraction layer already: adding stocks doesn't require redesigning anything upstream.

New Piece 1: Alpaca Adapter (stocks/alpaca_adapter.py)

Implements the same `BaseAdapter` interface as your crypto adapters:

```
python
class AlpacaAdapter(BaseAdapter):
    async def connect(self): ... # Alpaca's own websocket for live bars
    async def subscribe_candles(self, symbol, timeframe, callback): ...
    async def fetch_historical(self, symbol, timeframe, since, limit): ...
    async def place_order(self, symbol, side, size): ... # build last
```

Output normalized into the exact same candle schema as crypto data — the rest of your pipeline (features, regime, strategies, allocator, risk gate) doesn't need to know or care whether a pair is `BTC/USDT` or `AAPL`.

Note: US equities markets aren't 24/7 like crypto — your orchestrator needs to handle market-hours awareness for stock symbols (skip scanning/trading outside session hours) while crypto pairs keep running continuously.

New Piece 2: Reinforcement Learning Layer (regime/rl_optimizer.py)

Where RL fits: **not replacing your strategies, but tuning them.**

State: current regime probabilities + recent performance metrics (win rate, drawdown, avg trade outcome) per pair/strategy combination.

Action: adjust strategy parameters (e.g. moving-average lengths, confidence thresholds, position-size scaling) or adjust the allocator's weighting between strategies.

Reward function — this is where "save money > make money" gets encoded numerically:

```
python
reward = (profit_if_positive * 1.0) - (loss_if_negative * 3.0) - (drawdown_penalty)
```

Losses are weighted more heavily than equivalent gains — this directly punishes the RL agent more for losing than it rewards it for winning, which is what actually makes "capital preservation over profit" a trainable objective instead of just a rule bolted on top.

Suggested approach: start with a simple policy-gradient or even a basic Q-learning setup on discretized state/action spaces — don't reach for a deep RL framework until the simple version proves the concept. RL is notoriously easy to over-engineer before you have a working baseline.

Critical guardrail: RL-adjusted parameters must still pass through the Risk Gate and Bias Gate below before being applied live — RL improving a strategy in backtest doesn't mean it's safe to trust blindly, especially since RL is prone to overfitting to whatever data it was trained on.

New Piece 3: Backtesting Bias Gate (validation/bias_gate.py)

Runs whenever the RL layer proposes a strategy update, or when a new strategy is added — before it's allowed into paper or live trading:

Survivorship bias — check: is the backtest using historical pair/asset lists that exclude delisted/failed assets? Fix: use point-in-time universe data, not today's list applied retroactively.

Look-ahead bias — check: does any feature or signal use information not actually available at that point in time (e.g. using a day's closing price to make a decision earlier that day)? Fix: strict timestamp auditing on every feature.

Overfitting — check: does performance hold up on an out-of-sample period the model never trained on? Fix: mandatory train/validation/test split, reject if test performance diverges significantly from training performance.

Data-snooping bias — check: has this exact strategy configuration been tested and re-tested against the same historical data many times until something looked good? Fix: track how many variations were tried; heavily discount results from strategies born from extensive parameter searching on the same dataset.

Transaction-cost bias — check: does the backtest include realistic fees, slippage, and spread? Fix: never trust a backtest run with zero-cost assumptions; apply conservative cost estimates.

Selection bias — check: was this strategy cherry-picked from many tested strategies without correcting for how many were tried? Fix: apply a penalty proportional to the number of strategies tested before this one was chosen (similar concept to multiple-hypothesis correction).

```
python
def validate_strategy(strategy, backtest_results, metadata) -> dict:
    checks = {
        "survivorship_ok": check_survivorship(metadata),
        "lookahead_ok": check_lookahead(strategy),
        "overfitting_ok": check_oos_performance(backtest_results),
        "data_snooping_ok": check_iteration_count(metadata),
        "transaction_cost_ok": check_realistic_costs(backtest_results),
        "selection_bias_ok": check_selection_penalty(metadata),
    }
    return {"approved": all(checks.values()), "details": checks}
```

A strategy (or RL-tuned update) must pass ALL 6 checks before the Risk Gate even sees it.

Updated Build Order

Everything from the v1 build order (schema → adapters → aggregation → storage → features → regime model) — Binance first, prove it works.

Add `alpaca_adapter.py` using the same interface — prove stocks flow through identically to crypto.

Build one strategy, get it through the pipeline end-to-end on paper trading (as before).

Build the `bias_gate.py` — test it against a deliberately bad strategy (e.g. one that was overfit) to confirm it correctly rejects it.

Only once the static pipeline is proven end-to-end: add `rl_optimizer.py`, and route every RL-proposed update through `bias_gate.py` → `risk_gate.py` before it's live.

Expand to remaining exchanges, then remaining strategies, then scale to full pair/timeframe scanning across both asset classes.

RL and the bias gate are deliberately last — they're refinements on top of a working switching pipeline, not things to build before you have a baseline to improve.